

I040-2-8461

Open Source Software

03. Git and Github

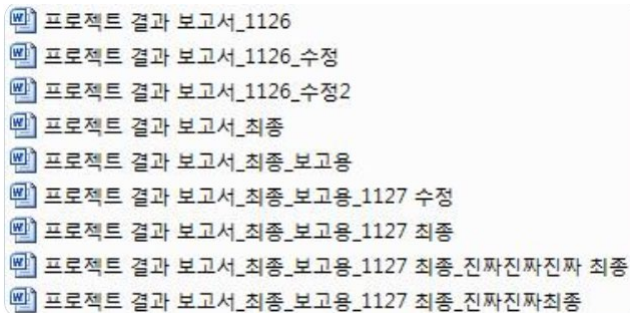
Dr. Kyudong Park

School of Information Convergence



버전 관리

- Git 은 버전을 관리하기 위한 도구
 - version control
- Git을 활용하지 않으면... (단순 저장 방식, 파일 이름 뒤에 version 추가하는 방식 등)
 - 변경 내역 확인이 어려움
 - 버전을 되돌리기 어려움
 - 협력하기 어려움



버전 관리

- 프로그래밍 / 코딩은 버전을 쌓아 올리는 것
 - 여러 버전이 쌓여서 소프트웨어가 만들어짐
- 개발에서의 버전 관리는
 - 누가, 어떻게 변경했는지에 대한 변경 내역들을 기억
 - 필요하다면 특정 시점의 버전으로 되돌릴 수 있음
 - 여러 명이 협업하는 과정에서 코드를 쉽게 나누고 합칠 수 있음
- 커뮤니티 단위 협업 위주의 오픈소스에서는 git이 매우 중요함

Contents

- [환경 설정](#)
- 버전 관리의 프로세스
- 비교 & 되돌리기 & 임시저장
- 브랜치의 개념
- Github를 활용한 협업

1. 환경 설정

- git 설치

 **git** --everything-is-local

Git is a **free and open source** distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is **easy to learn** and has a **tiny footprint with lightning fast performance**. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like **cheap local branching**, convenient staging areas, and **multiple workflows**.





About

The advantages of Git compared to other source control systems.



Documentation

Command reference pages, Pro Git book content, videos and other material.



Downloads

GUI clients and binary releases for all major platforms.



Community

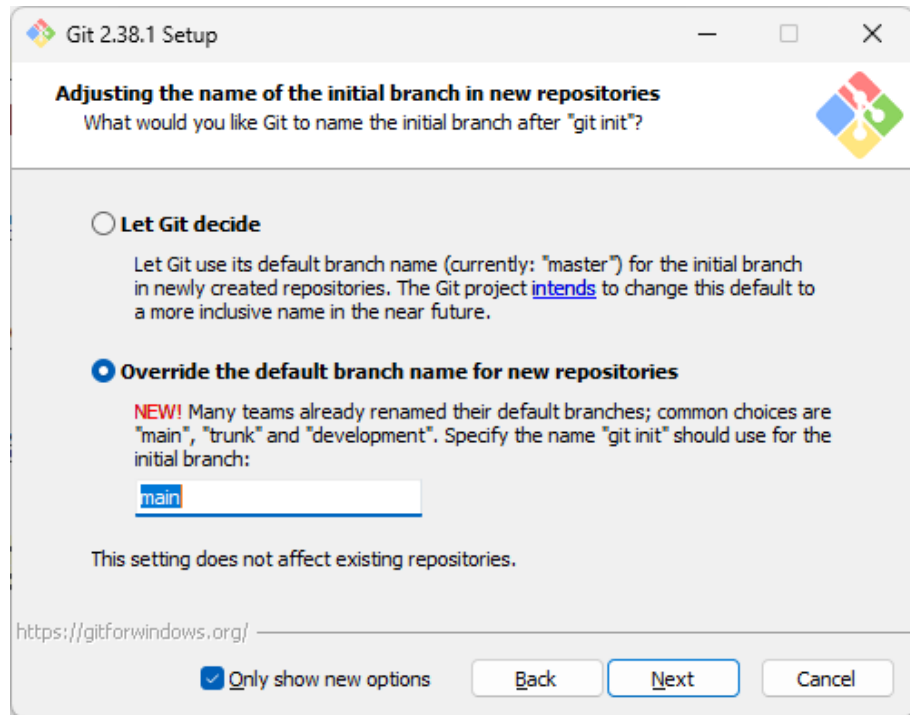
Get involved! Bug reporting, mailing list, chat, development and more.



Latest source Release
2.37.3
[Release Notes \(2022-08-30\)](#)
[Download for Windows](#)

1. 환경 설정

- git 설치
 - default branch name 변경 = main



1. 환경 설정

- 설치 후 default branch name 수동 변경
 - main 을 추천
 - replace usages of whitelist/blacklist and master/slave
 - 설정하기
 - `$ git config --global init.defaultBranch main`
 - 확인하기
 - `$ git config --get init.defaultBranch`

1. 환경 설정

- git bash
 - git 의 모든 버전에는 만든 사람의 이름, 이메일이 명시됨
 - 초기 설정을 해주어야 함
 - `$ git config --global user.name "user name"`
 - `$ git config --global user.email your_email`
 - 그 외 설정값의 확인
 - `$ git config --list`

```
kdpark@DESKTOP-78041M2 MINGW64 ~  
$ git config --list  
diff.astextplain.textconv=astextplain  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
http.sslbackend=openssl  
http.sslcainfo=C:/Program Files/Git/mingw64/ssl/certs/ca-bundle.crt  
core.autocrlf=true  
core.fscache=true  
core.symlinks=false  
pull.rebase=false  
credential.helper=manager-core  
credential.https://dev.azure.com.usehttppath=true  
init.defaultbranch=master  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
user.name=kdpark  
user.email=87839843+Dr-kdpark@users.noreply.github.com  
core.editor="C:\\Users\\kdpark\\AppData\\Local\\Programs\\Microsoft VS Code\\Code.exe" --wait
```

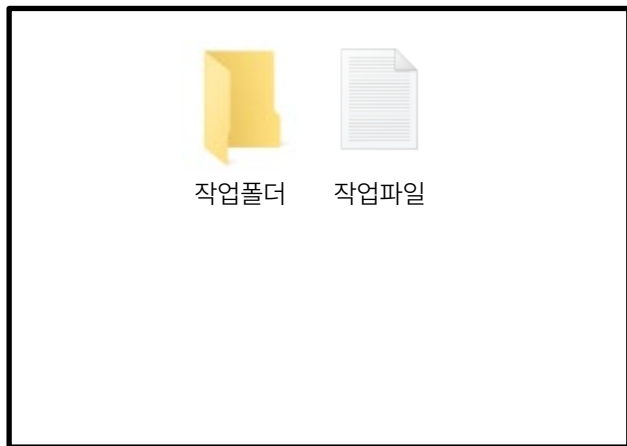

Contents

- 환경 설정
- 버전 관리의 프로세스
- 비교 & 되돌리기 & 임시저장
- 브랜치의 개념
- Github를 활용한 협업

2. 버전 관리의 프로세스

- git 에서 버전이 만들어지는 과정 1
 - 작업 디렉토리를 만들
 - 작업 디렉토리 내에서 작업을 수행함

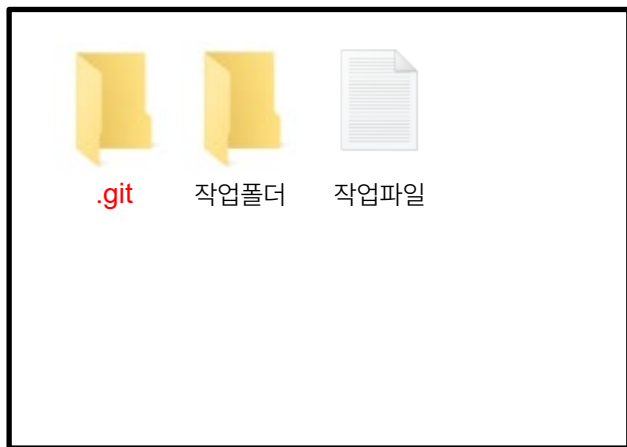
작업 디렉토리



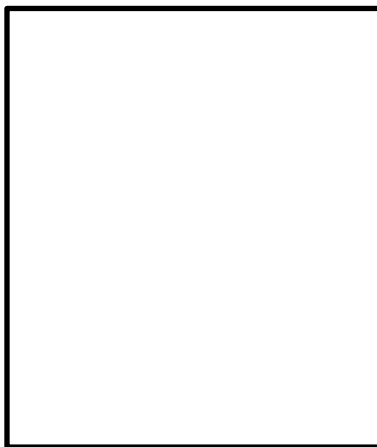
2. 버전 관리의 프로세스

- git 에서 버전이 만들어지는 과정 2
 - 버전을 만들다 = 특정 순간의 변경 사항을 기억한다
 - 초기 설정 완료 및 폴더와 파일 생성 (git init)

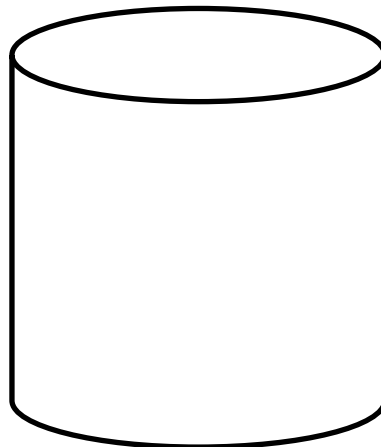
작업 디렉토리



스테이지 (Stage)

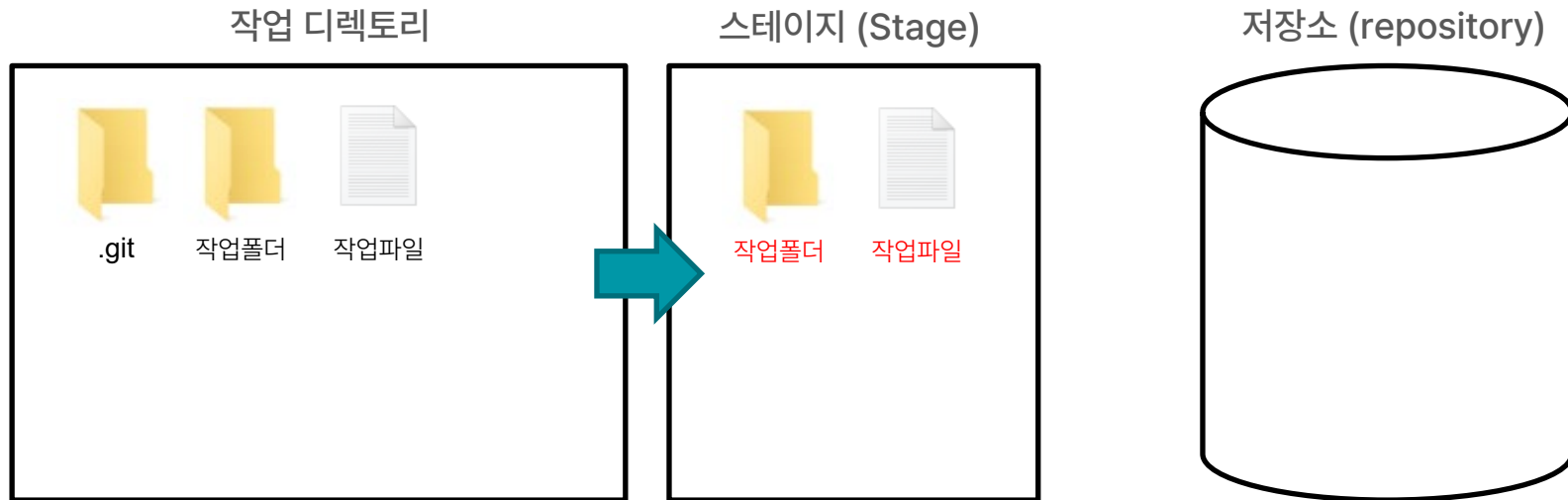


저장소 (repository)



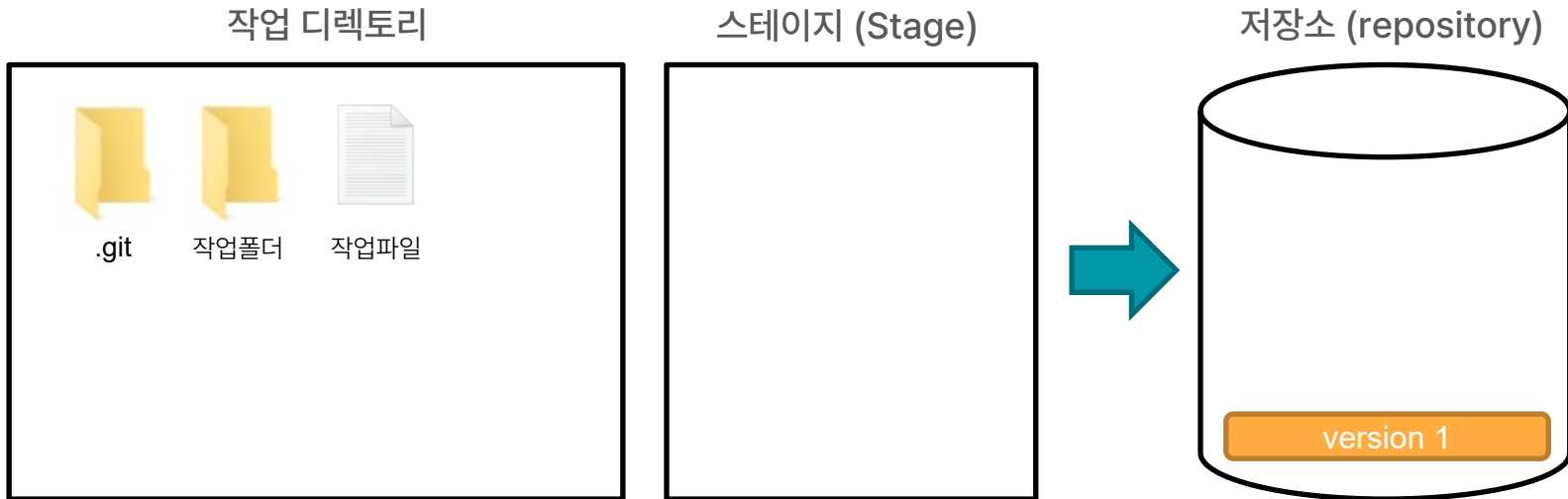
2. 버전 관리의 프로세스

- git 에서 버전이 만들어지는 과정 3
 - 다음 버전에 반영할 파일들을 스테이지에 올림 (git add)
 - 스테이지 = 다음 버전이 될 후보가 올라가는 공간 (명시적으로 보이지 않음)



2. 버전 관리의 프로세스

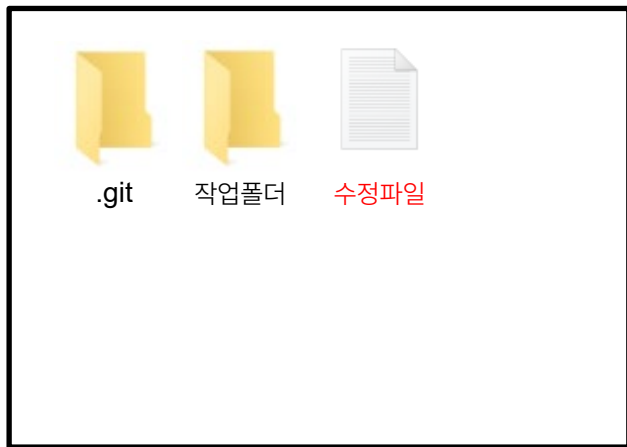
- git 에서 버전이 만들어지는 과정 4
 - 스테이지의 내용을 새로운 버전으로 올림 (git commit)
 - repository : commit 내용들이 쌓이는 공간



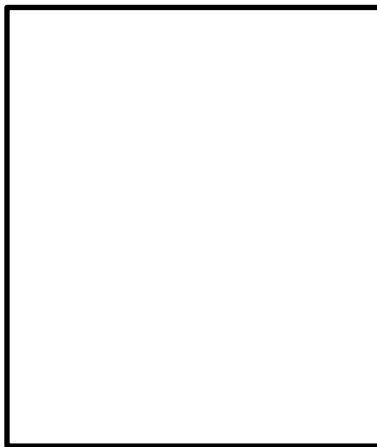
2. 버전 관리의 프로세스

- git 에서 버전이 만들어지는 과정 5
 - 파일이 수정될 때, 수정 여부를 tracking 함

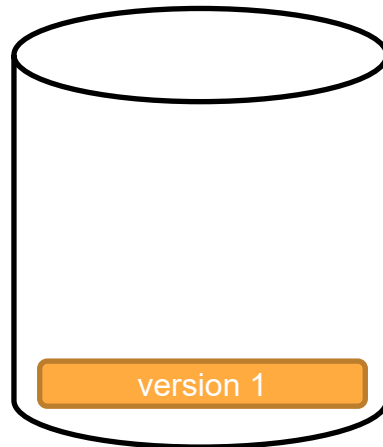
작업 디렉토리



스테이지 (Stage)

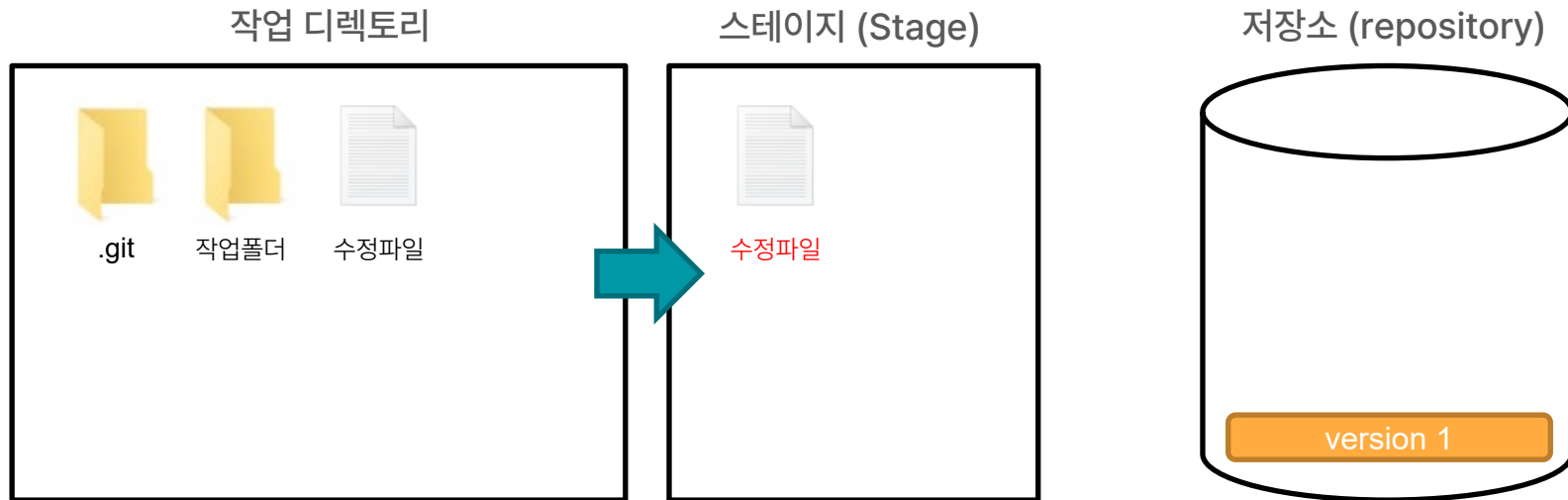


저장소 (repository)



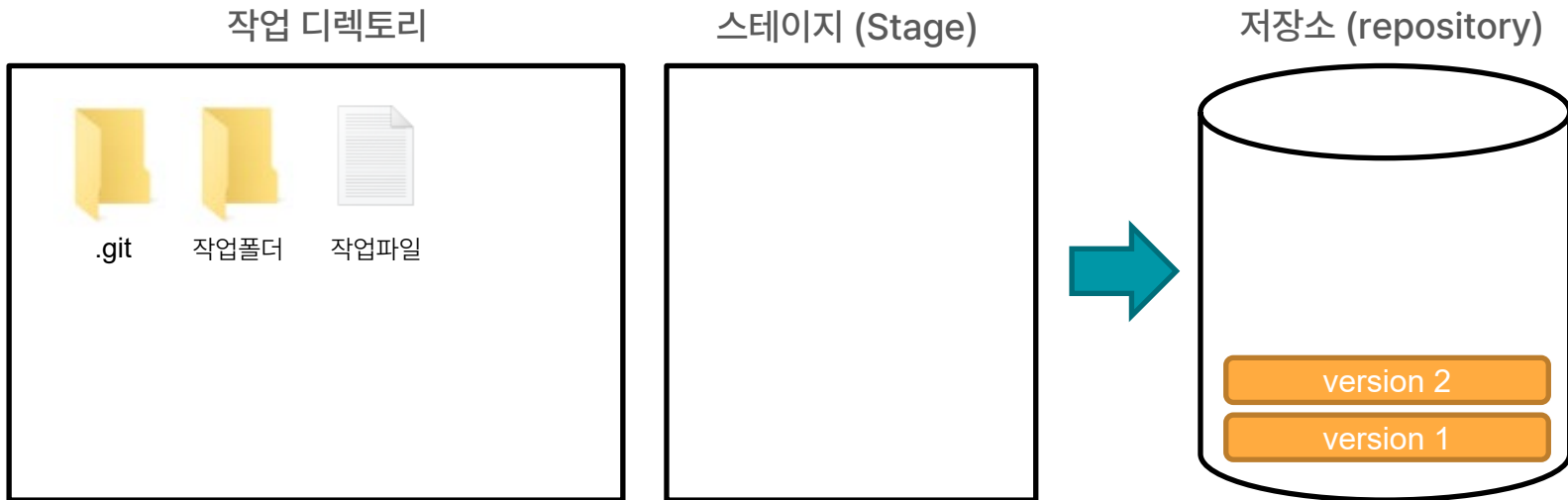
2. 버전 관리의 프로세스

- git 에서 버전이 만들어지는 과정 6
 - 수정 파일을 스테이지에 올림 (git add)



2. 버전 관리의 프로세스

- git 에서 버전이 만들어지는 과정 7
 - 스테이지의 내용을 새로운 버전으로 추가함 (git commit)

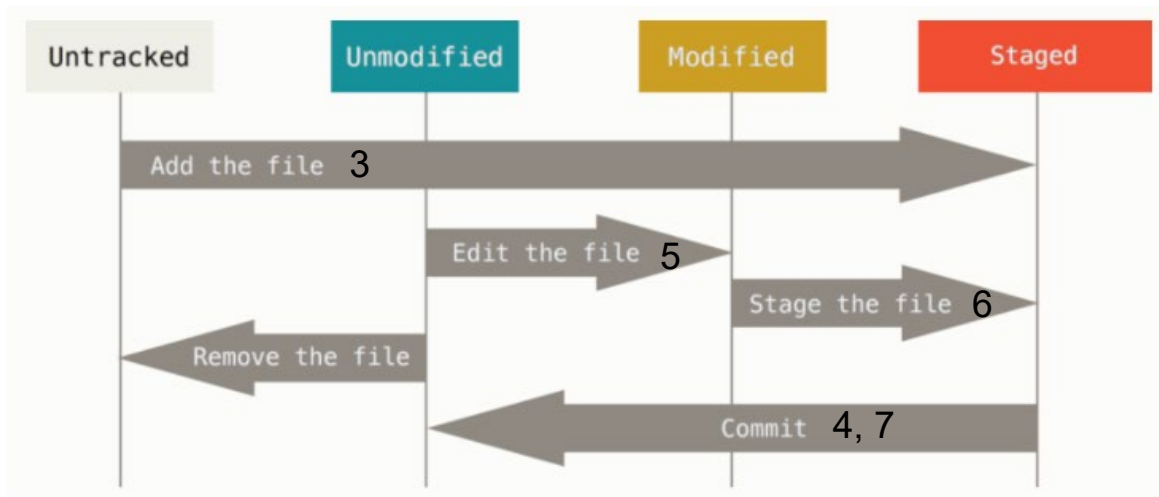


2. 버전 관리의 프로세스

- local에서의 git 관리 명령어 정리
 - git 초기화
 - `$ git init`
 - stage 에 올리기
 - `$ git add [file name]`
 - `$ git add .`
 - 전체 파일
 - local repository 에 올리기
 - `$ git commit -m [메시지]`
 - 메시지의 내용이 중요!
 - 작업 내용을 잘 파악할 수 있도록 작성

2. 버전 관리의 프로세스

- 파일의 상태 파악
 - `git status` : 현재 파일에 대한 상태를 알려줌



2. 버전 관리의 프로세스

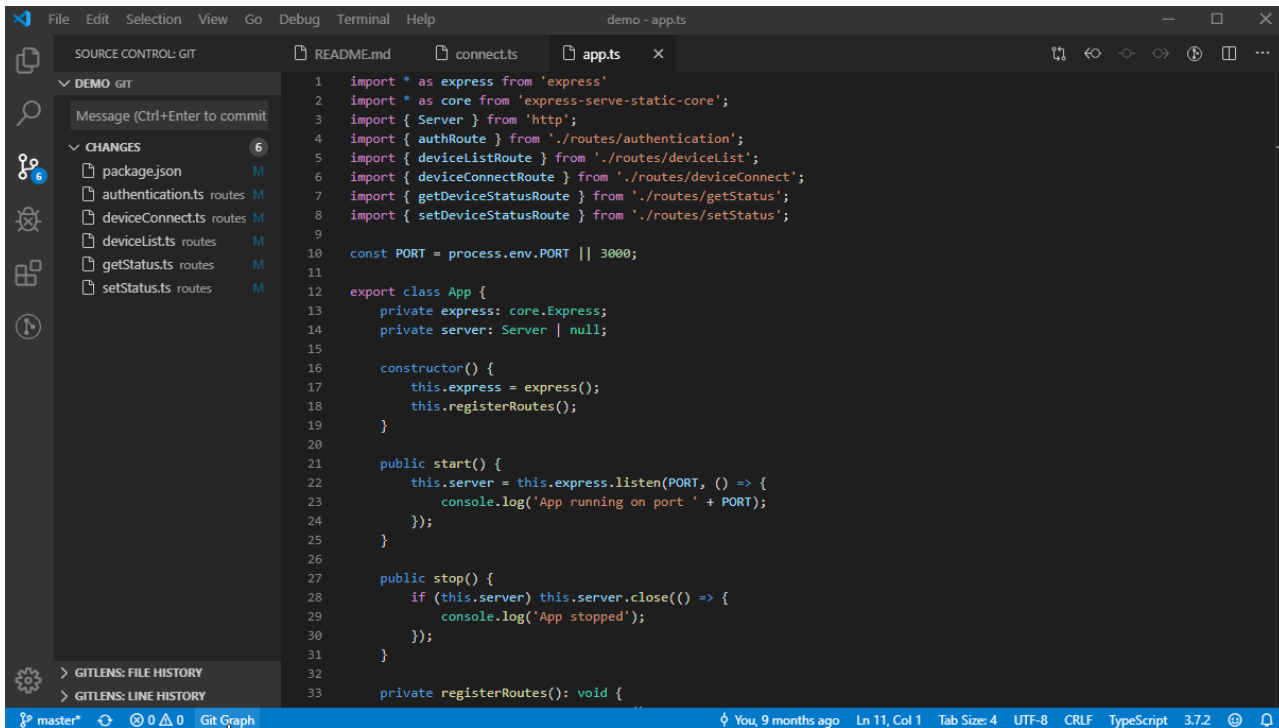
- 이력 조회
 - git log : 커밋에 대한 조회
 - git log --oneline : 단순 조회
 - git log --patch : 각각의 커밋의 변경내역을 상세히 표시
 - git log --graph : 커밋을 그래프 형태로 출력
 - commit 기록에 대한 hash 가 만들어짐, 가장 최근의 commit 의 포인터는 HEAD

```
$ git log
commit cbd6274b06aa2ad31e9fc4f4705143740a1c46ab (HEAD -> main,
origin/main, origin/HEAD)
Author: [redacted] Lee <[redacted]@naver.com>
Date:   Wed Mar 27 16:33:29 2024 +0900

    feat: 기존 DB 유저 패스워드 암호화 로직 삭제
```

2. 버전 관리의 프로세스

- 이력 조회
 - VS code 에서 git graph extensions 활용



2. 버전 관리의 프로세스

- 커밋에 대한 별칭 추가
 - `git tag [tag명] [commit명]`
 - `commit명` 생략 시, 가장 최근 `commit` 의 별칭으로 추가
 - `git tag -l` : 태그 조회하기 (`--list`)
 - `git tag -d [tag명]` : 태그 삭제하기 (`--delete`)

2. 버전 관리의 프로세스

- 관리되지 않을 파일을 미리 설정해두기
 - .gitignore
 - node 의 사례
 - 소스코드 저장소에 넣지 않는 파일
 - 외부로부터 설치되는 라이브러리
 - 환경 변수 설정 파일 등

```
3  # dependencies
4  /node_modules
5  /.pnp
6  .pnp.js
7
8  # testing
9  /coverage
10
11 # production
12 /build
13
14 # misc
15 .DS_Store
16 .env.local
17 .env.development.local
18 .env.test.local
19 .env.production.local
20
21 npm-debug.log*
22 yarn-debug.log*
23 yarn-error.log*
```

2. 버전 관리의 프로세스

- git 실습
 - git 에 의해 관리되는 폴더 환경 구축
 - 파일 1개 생성 후 내용 작성
 - stage 올리기
 - 현재 git 파일 상태 보기
 - 커밋하기
 - 파일 내용 변경
 - 현재 git 파일 상태 보기
 - 커밋하기
 - 최근 커밋에 대해 v1.0.0 태그 추가하기
 - 커밋에 대한 내역 보기

Contents

- 환경 설정
- 버전 관리의 프로세스
- 비교 & 되돌리기 & 임시저장
- 브랜치의 개념
- Github를 활용한 협업

3. 비교 & 되돌리기 & 임시저장

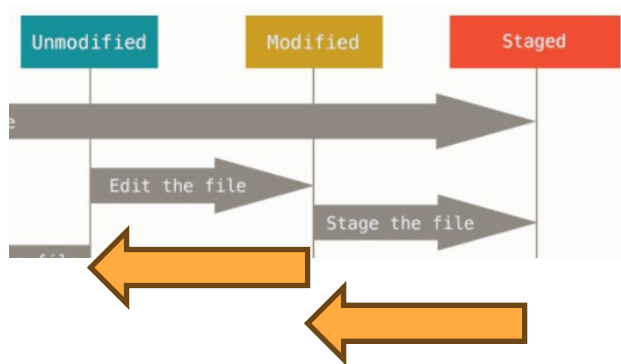
- 버전 비교하기
 - `git diff` : 최근 커밋과 작업 디렉터리 비교하기
 - `git diff --staged` : 최근 커밋과 스테이지 비교하기
 - `git diff [commit1] [commit2]` : 커밋1을 기준으로 커밋2에서 달라진 점 비교하기
 - commit 명 참고 : `HEAD^^` 또는 `HEAD~2` (^의 개수 또는 ~ 숫자 만큼 이전 커밋을 가리킴)

```
kdpark@DESKTOP-78041M2 MINGW64 ~/oss-git (master)
$ git diff
warning: LF will be replaced by CRLF in hello.txt.
The file will have its original line endings in your working directory
diff --git a/hello.txt b/hello.txt
index 7d1d98f..fdcc277 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1,3 +1,5 @@
 hello~?

+world~?
+
```

3. 비교 & 되돌리기 & 임시저장

- 작업 되돌리기 (파일 단위)
 - restore : 파일 상태를 되돌리기
 - `git restore [filename]`
 - unmodifying a modified file
 - 최근 커밋으로 파일을 복구 (modified 상태를 되돌림, 전체 파일은 .)
 - `git restore --staged [filename]`
 - unstaging a staged files
 - staging 에 올라간 파일 다시 unstaging 시키기
 - `git restore --source=[commit hash] [filename]`
 - 특정 commit 으로 파일을 되돌림



3. 비교 & 되돌리기 & 임시저장

- 작업 되돌리기 (commit 단위)
 - revert : 되돌아간 상태에 대한 새로운 커밋을 추가하는 방식
 - A - B - C - D 흐름의 커밋에서 revert 하면 A - B - C - D - C 가 됨
 - git revert [취소할 커밋]
 - 취소하는 새로운 커밋을 만들기 때문에, commit message 를 작성해야함

3. 비교 & 되돌리기 & 임시저장

- 작업 되돌리기

- reset : 되돌아가는 버전의 시점으로 되돌아가며, 되돌린 이후 버전 (commit 내역) 은 삭제됨
 - soft reset : commit 을 삭제하고, 스테이지에 올라가있는 상태로 되돌림 (스테이지와 변경내역은 유지)
 - mixed reset : commit, 스테이지를 삭제함. (작업폴더의 변경 내역은 유지)
 - hard reset : 되돌아가는 버전의 commit 이 완료된 상태로 통째로 되돌림 (변경 내역도 삭제함)
- git reset [되돌아갈 커밋] : 예전 커밋으로 돌아가기 (mixed reset)
 - git reset --soft [되돌아갈 커밋]
 - git diff --staged 명령어로 스테이지에 내역이 남아있는지 확인
 - git reset --hard [되돌아갈 커밋]

3. 비교 & 되돌리기 & 임시저장

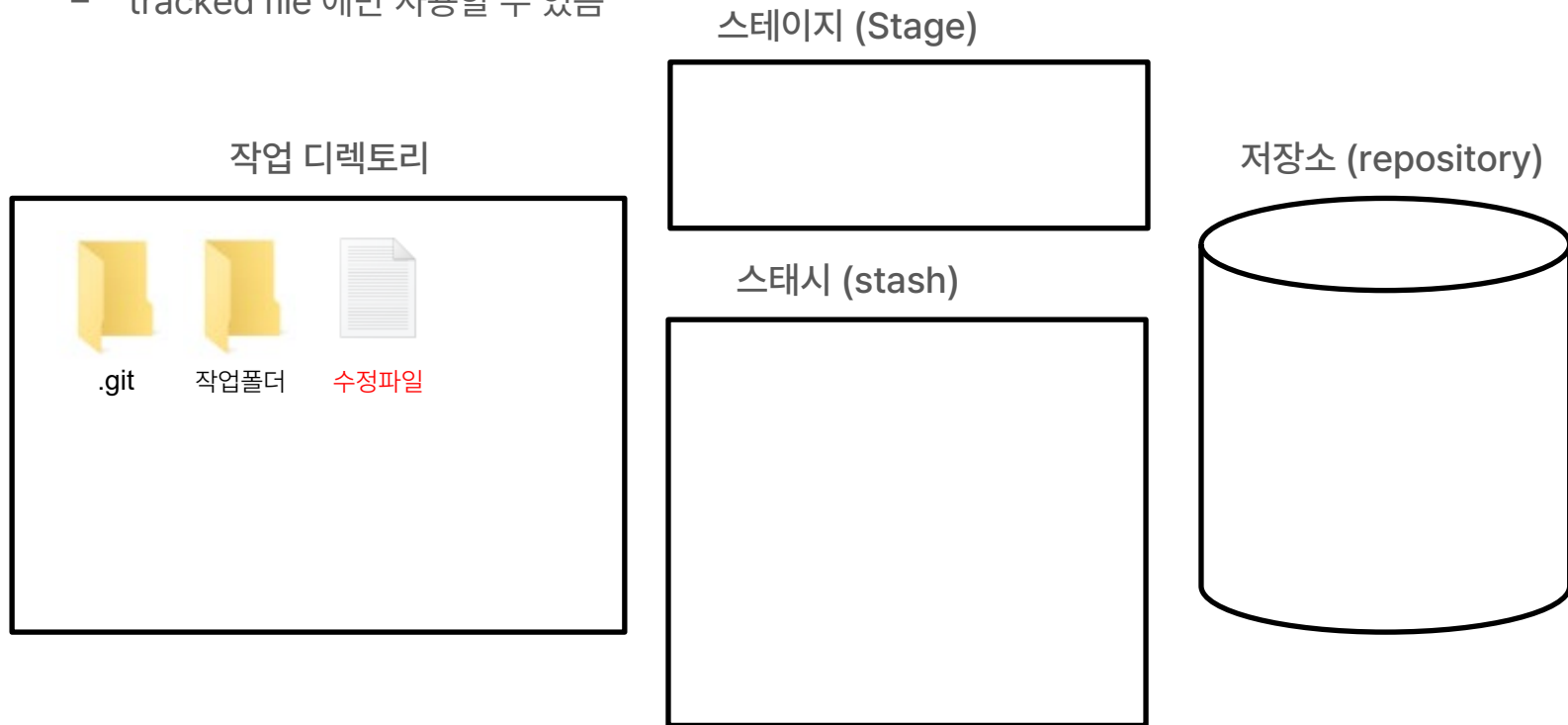
- 임시 저장하기
 - 작업 내역이 아직 commit 에 올릴만한 수준은 아니지만, 그렇다고 버리기는 아까울 때
 - 작업 도중에 갑자기 더 급한 부분을 먼저 수정해야할 때
 - 작업 도중에 고양이가 일을 못 하게 방해할 때



- 변경 내역을 모두 지워버리는 것 보다는 임시저장 시키는 것이 바람직함

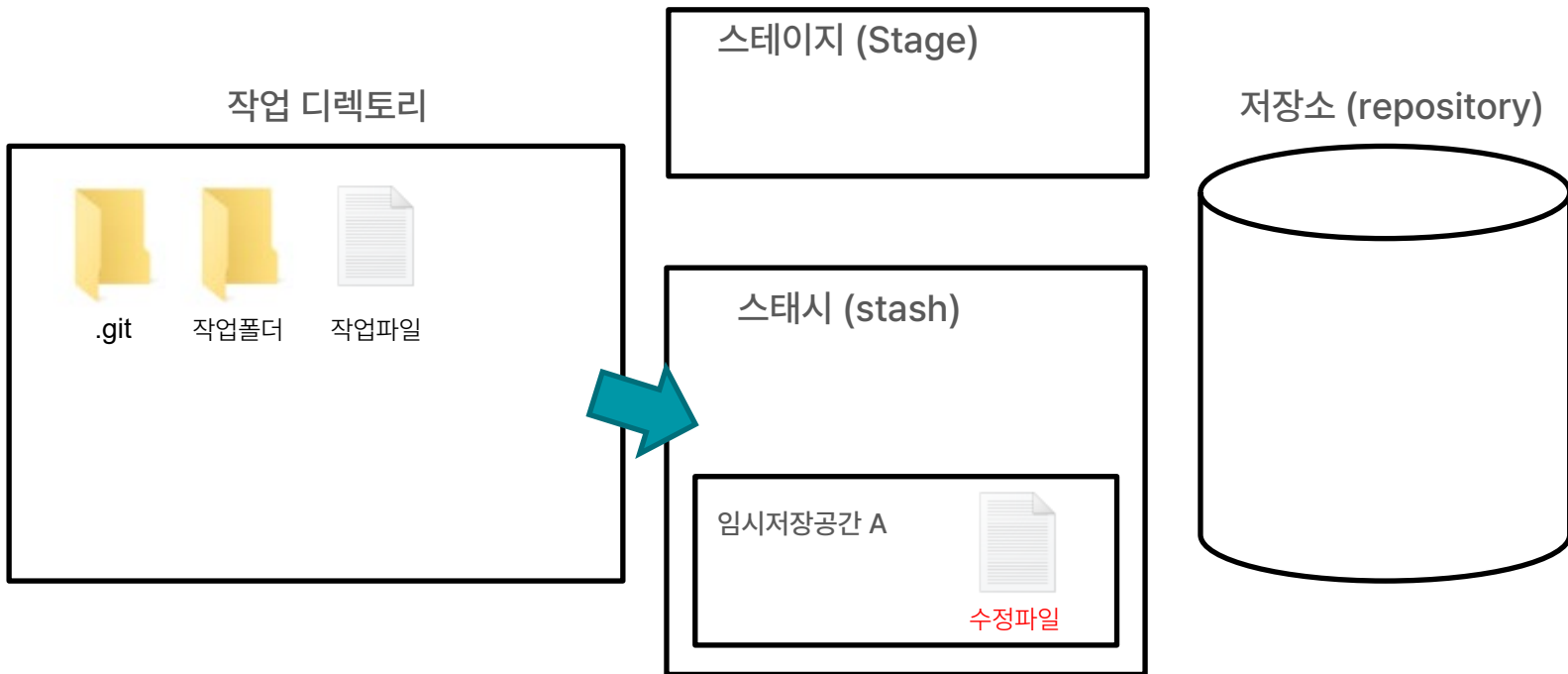
3. 비교 & 되돌리기 & 임시저장

- 스테시 (stash)
 - tracked file에만 사용할 수 있음



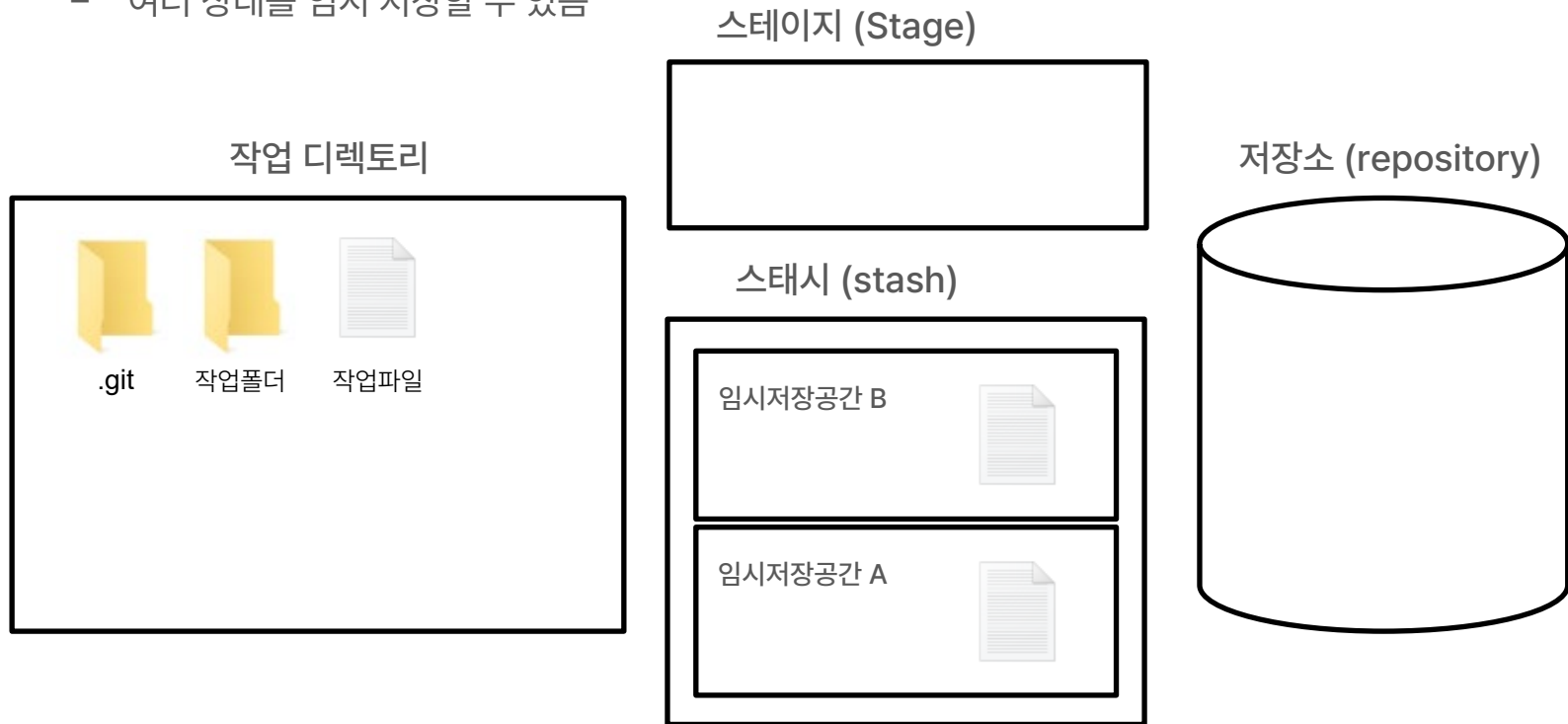
3. 비교 & 되돌리기 & 임시저장

- 스테시 (stash)
 - `git stash` : 현재 수정을 임시 저장하고 이전으로 돌아감 (-m 을 통해 메시지 저장 가능)



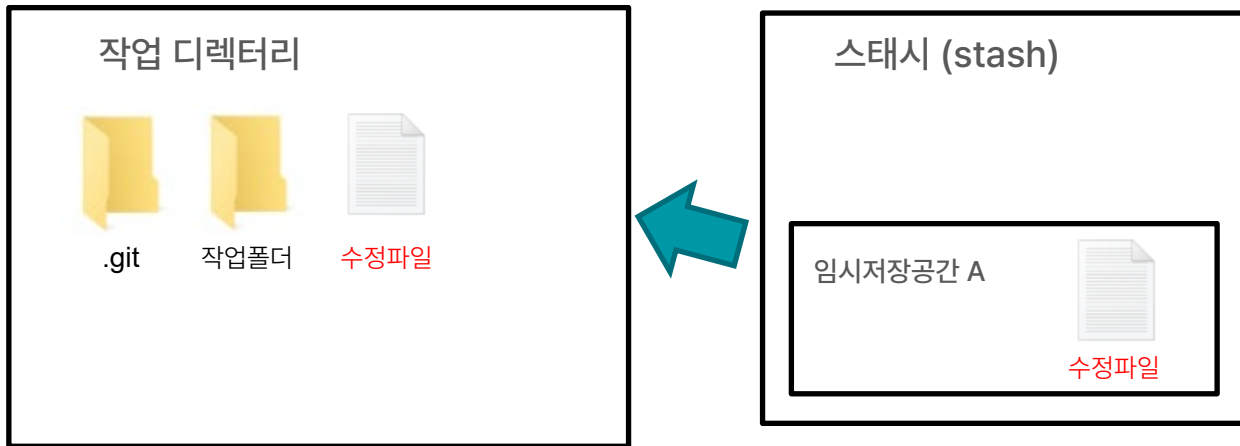
3. 비교 & 되돌리기 & 임시저장

- 스테시 (stash)
 - 여러 상태를 임시 저장할 수 있음



3. 비교 & 되돌리기 & 임시저장

- 임시 저장된 작업 내역 조회하기
 - `git stash list`
- 임시 저장된 작업을 작업 디렉터리에 적용하기
 - `git stash apply [스태시이름]`
 - 스태시 이름 예시 : `stash@{0}`



3. 비교 & 되돌리기 & 임시저장

- 임시 저장된 작업을 삭제하기
 - `git stash drop [스태시이름]`
- 모든 임시 저장 작업을 삭제하기
 - `git stash clear`

작업 디렉터리



.git



작업폴더



수정파일

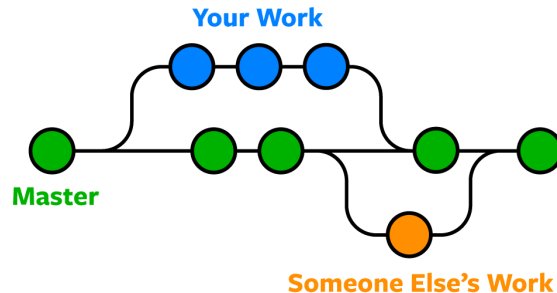
스태시 (stash)

Contents

- 환경 설정
- 버전 관리의 프로세스
- 비교 & 되돌리기 & 임시저장
- 브랜치의 개념
- Github를 활용한 협업

4. 브랜치의 개념

- branch : 나뉘어가지
 - 버전을 여러 흐름으로 나누어서 관리하는 방법
- branch의 필요성
 - 각자 기능을 맡아서 따로 개발하는 경우
 - 합치는 부분을 일일이 대조, 서로의 작업과 관련 없는 부분들도 확인해야함
 - 새로운 요구사항을 반영한 다양한 버전
 - 고객 별로 요구사항이 다를 때

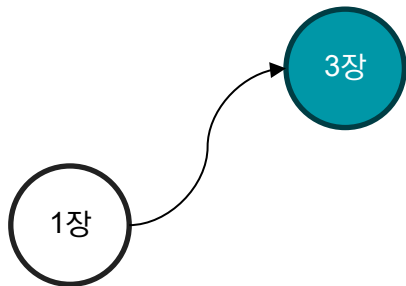


4. 브랜치의 개념

- branch 를 활용한 작업
 - 현재 1장만 완료된 보고서, A 가 2장 작업하기로 하고 B가 3장을 작업하기로 함
 - 시간이 없어서 동시에 시작함
 - B가 먼저 완성하고 기다림
 - A가 완성함
 - B가 1,2,3장을 합치고 있음
 - 그때 A가 2장에 고칠 부분을 발견하고 수정함
 - B는 1,2,3장을 합치는 것을 완료함
 - A가 B에게 수정할 부분을 알려줌
 - B가 갖고 있는 최종파일에서 2장을 고침
 - 결과물을 A와 나눠갖고 완성

4. 브랜치의 개념

- branch 를 활용한 작업



B가 먼저 완성하고 기다림

A가 완성함

B가 1,2,3장을 합치고 있음

그때 A가 2장에 고칠 부분을 발견하고 수정함

B는 1,2,3장을 합치는 것을 완료함

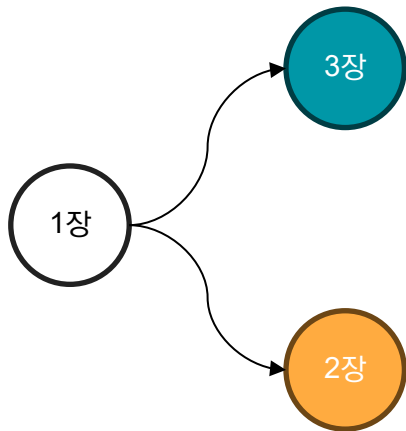
A가 B에게 수정할 부분을 알려줌

B가 갖고 있는 최종파일에서 2장을 고침

결과물을 A와 나눠갖고 완성

4. 브랜치의 개념

- branch 를 활용한 작업



B가 먼저 완성하고 기다림

A가 완성함

B가 1,2,3장을 합치고 있음

그때 A가 2장에 고칠 부분을 발견하고 수정함

B는 1,2,3장을 합치는 것을 완료함

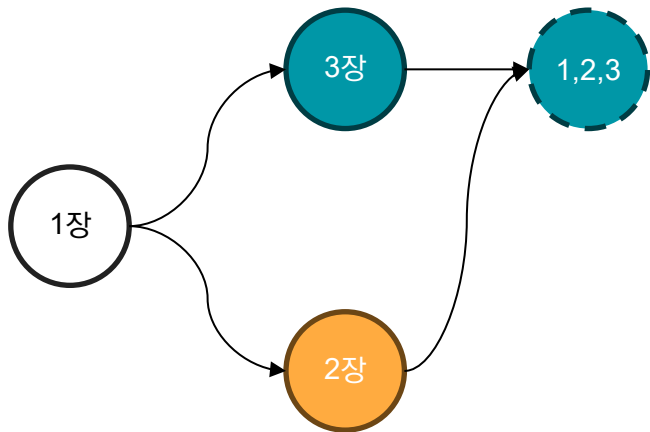
A가 B에게 수정할 부분을 알려줌

B가 갖고 있는 최종파일에서 2장을 고침

결과물을 A와 나눠갖고 완성

4. 브랜치의 개념

- branch 를 활용한 작업



B가 먼저 완성하고 기다림

A가 완성함

B가 1,2,3장을 합치고 있음

그때 A가 2장에 고칠 부분을 발견하고 수정함

B는 1,2,3장을 합치는 것을 완료함

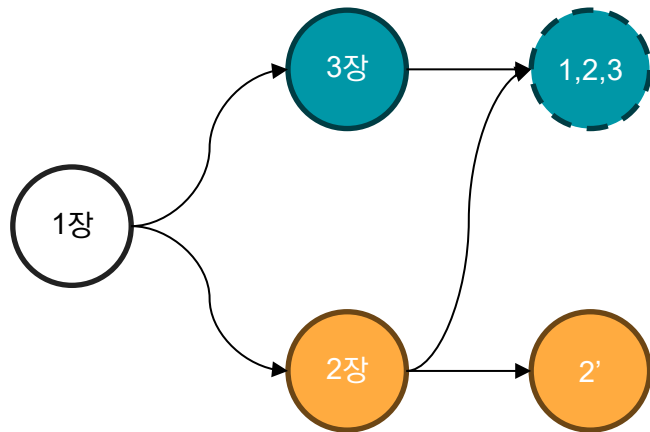
A가 B에게 수정할 부분을 알려줌

B가 갖고 있는 최종파일에서 2장을 고침

결과물을 A와 나눠갖고 완성

4. 브랜치의 개념

- branch 를 활용한 작업



B가 먼저 완성하고 기다림

A가 완성함

B가 1,2,3장을 합치고 있음

그때 A가 2장에 고칠 부분을 발견하고 수정함

B는 1,2,3장을 합치는 것을 완료함

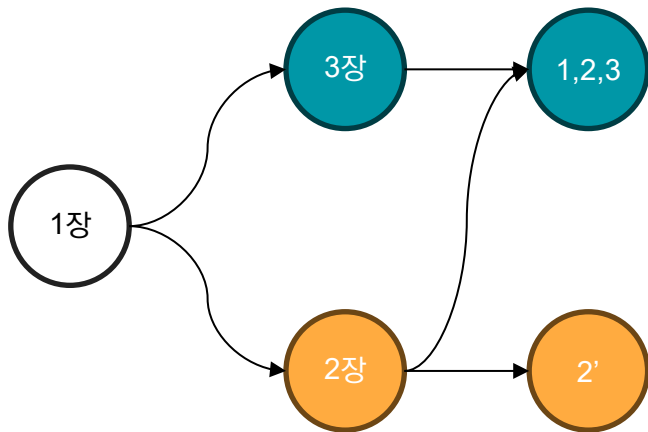
A가 B에게 수정할 부분을 알려줌

B가 갖고 있는 최종파일에서 2장을 고침

결과물을 A와 나눠갖고 완성

4. 브랜치의 개념

- branch 를 활용한 작업



B가 먼저 완성하고 기다림

A가 완성함

B가 1,2,3장을 합치고 있음

그때 A가 2장에 고칠 부분을 발견하고 수정함

B는 1,2,3장을 합치는 것을 완료함

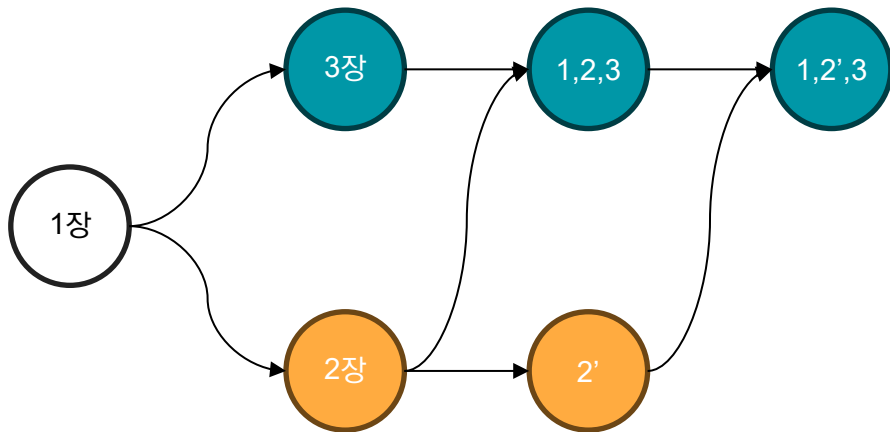
A가 B에게 수정할 부분을 알려줌

B가 갖고 있는 최종파일에서 2장을 고침

결과물을 A와 나눠갖고 완성

4. 브랜치의 개념

- branch 를 활용한 작업



B가 먼저 완성하고 기다림

A가 완성함

B가 1,2,3장을 합치고 있음

그때 A가 2장에 고칠 부분을 발견하고 수정함

B는 1,2,3장을 합치는 것을 완료함

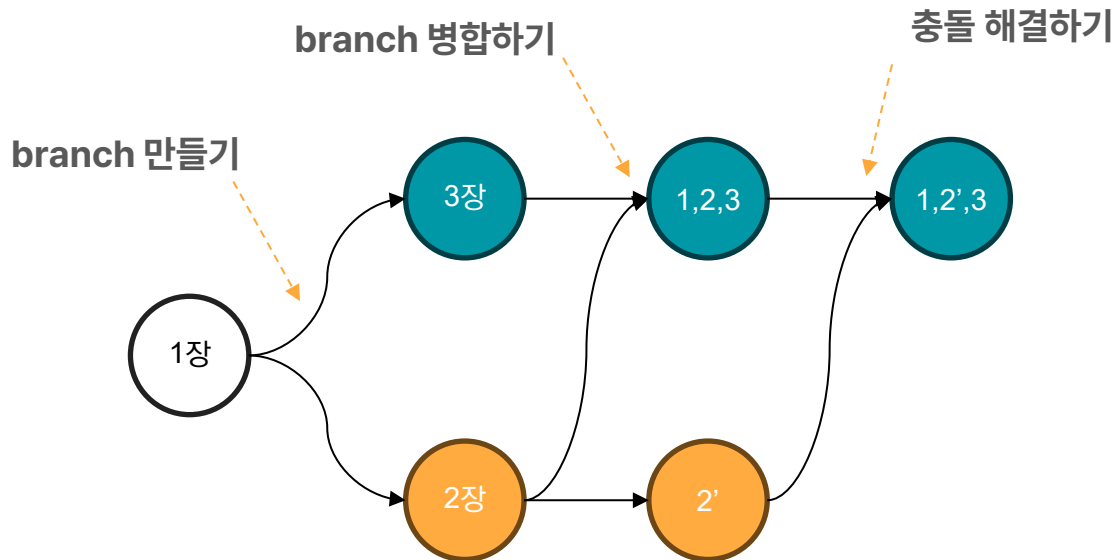
A가 B에게 수정할 부분을 알려줌

B가 갖고 있는 최종파일에서 2장을 고침

결과물을 A와 나눠갖고 완성

4. 브랜치의 개념

- branch 를 활용한 작업



4. 브랜치의 개념

- branch 확인하기
 - git branch : 현재 작업 중인 branch 확인하기

```
kdpark@DESKTOP-78041M2 MINGW64 ~/oss-git (master)
$ git branch
* master
```

- branch 만들기
 - git branch [branch명] : 브랜치 나누기

4. 브랜치의 개념

- branch 전환하기
 - git switch [branch명] : 특정 브랜치로 전환하기
 - 브랜치에 기록된 내용으로 작업 디렉터리의 내용이 전환됨
 - git switch featureA
- branch 병합하기
 - git merge [병합할 branch]
 - [A] branch 를 [main] branch 에 병합하고 싶을 때 = main branch 로 switch 한 뒤 git merge A

4. 브랜치의 개념

- 충돌 해결
 - 서로 다른 branch가 동일한 파일을 다르게 수정하였을 때, 이를 merge 하려고 하면 발생

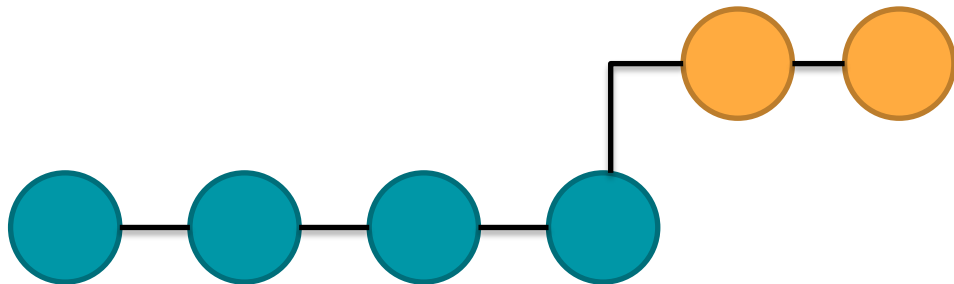
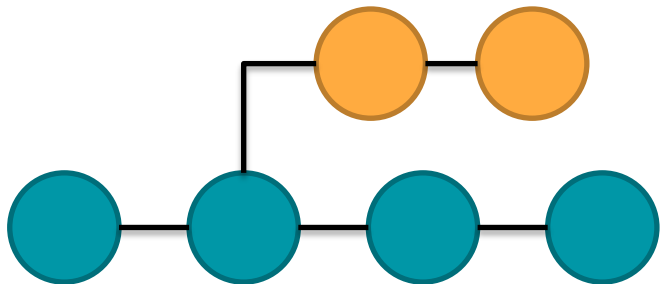
```
kdpark@DESKTOP-78041M2 MINGW64 ~/oss-git (master)
$ git merge hotfix
Auto-merging hello.txt
CONFLICT (content): Merge conflict in hello.txt
Automatic merge failed; fix conflicts and then commit the result.
```

```
kdpark@DESKTOP-78041M2 MINGW64 ~/oss-git (master|MERGING)
$ cat hello.txt
<<<<<<< HEAD
hello~??

world??
=====
hello
world
>>>>>>> hotfix
```

4. 브랜치의 개념

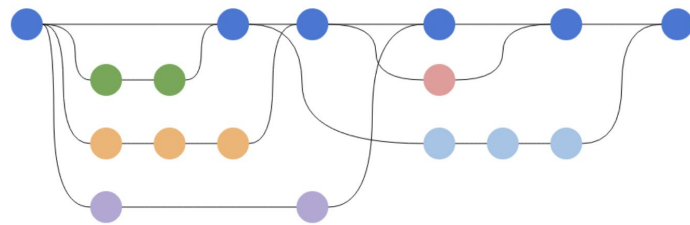
- 브랜치의 재배치
 - `git rebase [branch 명]`
 - 현재 활성화된 branch 가 뺏어나온 기준점을 입력한 branch 로 변경하는 방법
 - 마찬가지로 충돌 해결이 필요할 수 있음



4. 브랜치의 개념

- rebase 의 장점

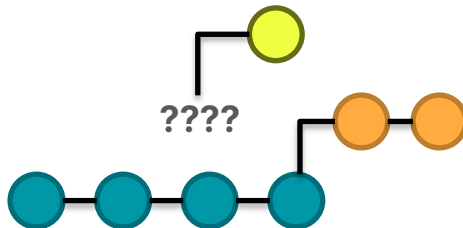
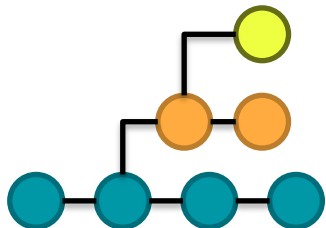
- commit 기록을 linear 하게 관리할 수 있음
- merge conflict commit 이 없어짐
- 점차 도입이 늘어나는 추세



복잡한 merge 의 결과

- 주의사항

- rebase 하려는 branch 를 다른 사람이 수정 중이라면...
- 혼자 작업하는 branch 에서만 활용!



4. 브랜치의 개념

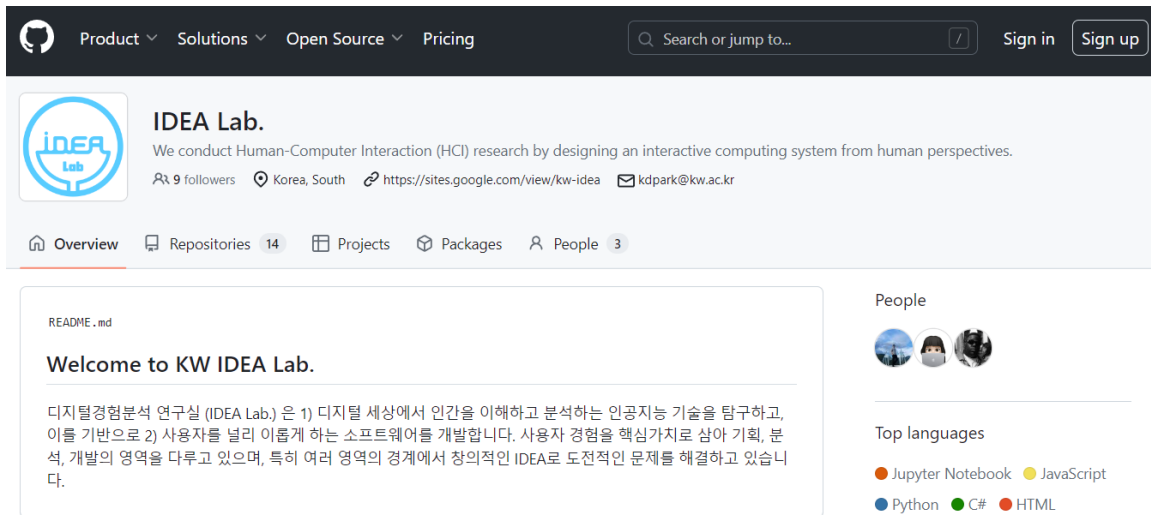
- 브랜치 삭제
 - `git branch -d [branch 명]`
 - 삭제를 위해서는 다른 branch 로 switch 해야함

Contents

- 환경 설정
- 버전 관리의 프로세스
- 비교 & 되돌리기 & 임시저장
- 브랜치의 개념
- [Github를 활용한 협업](#)

5. Github를 활용한 협업 - 환경 설정

- github
 - 원격 저장소 호스팅 서비스
 - git 으로 버전을 관리하는 프로젝트들이 모여있는 웹사이트
 - 개발자들의 SNS



The screenshot shows the GitHub profile page for 'IDEA Lab.'. The header includes the GitHub logo, navigation links (Product, Solutions, Open Source, Pricing), a search bar, and 'Sign in' and 'Sign up' buttons. The profile section features the 'iDEA Lab.' logo, the name 'IDEA Lab.', a bio 'We conduct Human-Computer Interaction (HCI) research by designing an interactive computing system from human perspectives.', 9 followers, location 'Korea, South', website 'https://sites.google.com/view/kw-idea', and email 'kdpark@kw.ac.kr'. Below this are tabs for Overview (selected), Repositories (14), Projects, Packages, and People (3). The 'Overview' tab shows a 'README.md' file with the text 'Welcome to KW IDEA Lab.' and a paragraph about digital experience analysis research. The 'People' section shows three profile pictures, and the 'Top languages' section shows a bar chart for Jupyter Notebook, JavaScript, Python, C#, and HTML.

Product Solutions Open Source Pricing Search or jump to... Sign in Sign up

IDEA Lab.
We conduct Human-Computer Interaction (HCI) research by designing an interactive computing system from human perspectives.
9 followers Korea, South <https://sites.google.com/view/kw-idea> kdpark@kw.ac.kr

Overview Repositories 14 Projects Packages People 3

README.md

Welcome to KW IDEA Lab.

디지털경험분석 연구실 (IDEA Lab.) 은 1) 디지털 세상에서 인간을 이해하고 분석하는 인공지능 기술을 탐구하고, 이를 기반으로 2) 사용자를 널리 이롭게 하는 소프트웨어를 개발합니다. 사용자 경험을 핵심가치로 삼아 기획, 분석, 개발의 영역을 다루고 있으며, 특히 여러 영역의 경계에서 창의적인 IDEA로 도전적인 문제를 해결하고 있습니다.

People

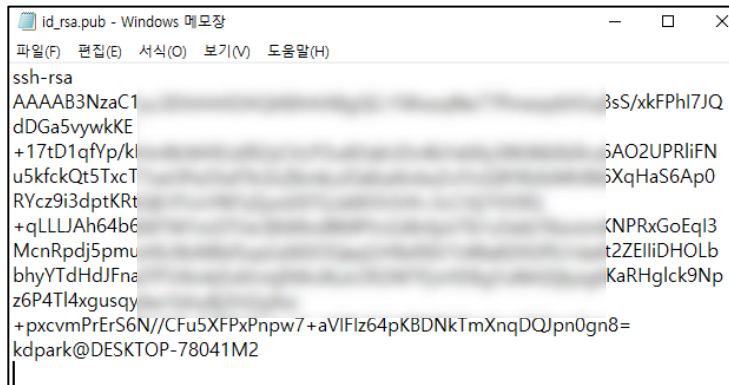
Top languages

Jupyter Notebook JavaScript Python C# HTML

5. Github를 활용한 협업 - 환경 설정

- 로컬/서버에서 github 에 SSH 로 연결하기
 - \$ ssh-keygen

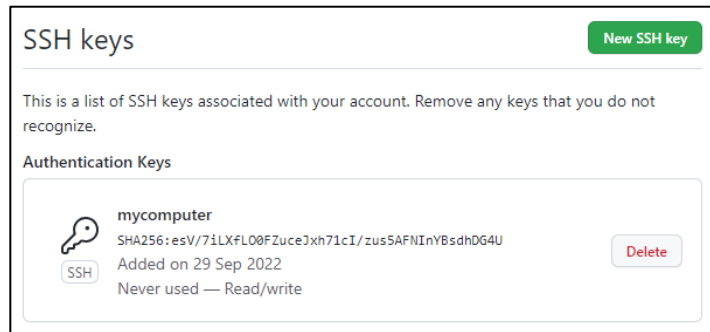
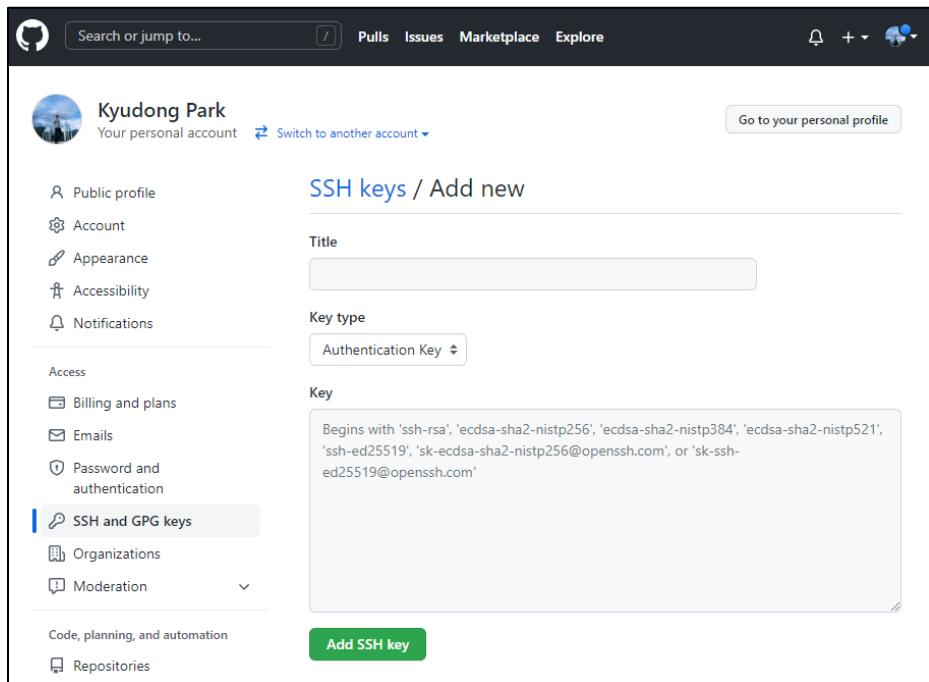
```
kdpark@DESKTOP-78041M2 MINGW64 ~  
$ ssh-keygen  
Generating public/private rsa key pair.  
Enter file in which to save the key (/c:/Users/kdpark/.ssh/id_rsa):  
Enter passphrase (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in /c:/Users/kdpark/.ssh/id_rsa  
Your public key has been saved in /c:/Users/kdpark/.ssh/id_rsa.pub  
The key fingerprint is:  
SHA256:esV/7iLXfL00FZuceJxh71cI/zus5AFNInYBsdhDG4U kdpark@DESKTOP-78041M2  
The key's randomart image is:  
+---[RSA 3072]-----+  
|  
|      ==o      |  
|    +E+  .    |  
|   . B o .   |  
|    ..+ =    |  
|   S o. + =   |  
|  . . . . B X |  
| . . . . *o@+ |  
| . . . . =o* =*|  
|    o  +=B    |  
+---[SHA256]-----+
```



```
id_rsa.pub - Windows 메모장  
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)  
ssh-rsa  
AAAAB3NzaC1  
dDGa5vywkE  
+17tD1qfYp/kl  
u5kfcKQt5TxcT  
RYcz9i3dptKRt  
+qLLUjAh64b6  
McnRpdj5pmu  
bhyYTdHdJFna  
z6P4Tl4xgusqy  
+pxcvmPrErS6N//CFu5XFPxPnpw7+aVIFlz64pKBDNkTmXnqDQJpn0gn8=  
kdpark@DESKTOP-78041M2
```

5. Github를 활용한 협업 - 환경 설정

- 로컬/서버에서 github 에 SSH 로 연결하기
 - 키 등록하기

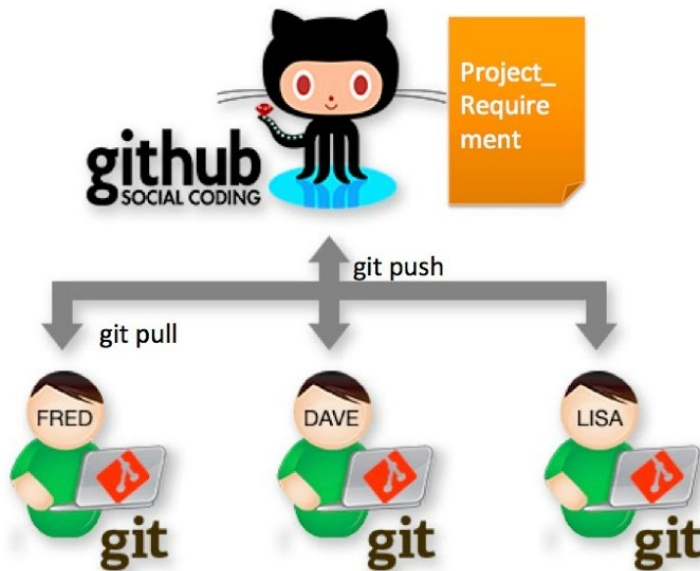


5. Github를 활용한 협업

- Github
 - 원격 저장소 호스팅 서비스
 - linux 의 소스코드 : <https://github.com/torvalds/linux>
 - git 의 소스코드 : <https://github.com/git/git>
 - 이외에도 수많은 오픈소스들의 코드를 확인할 수 있음
- Why github?

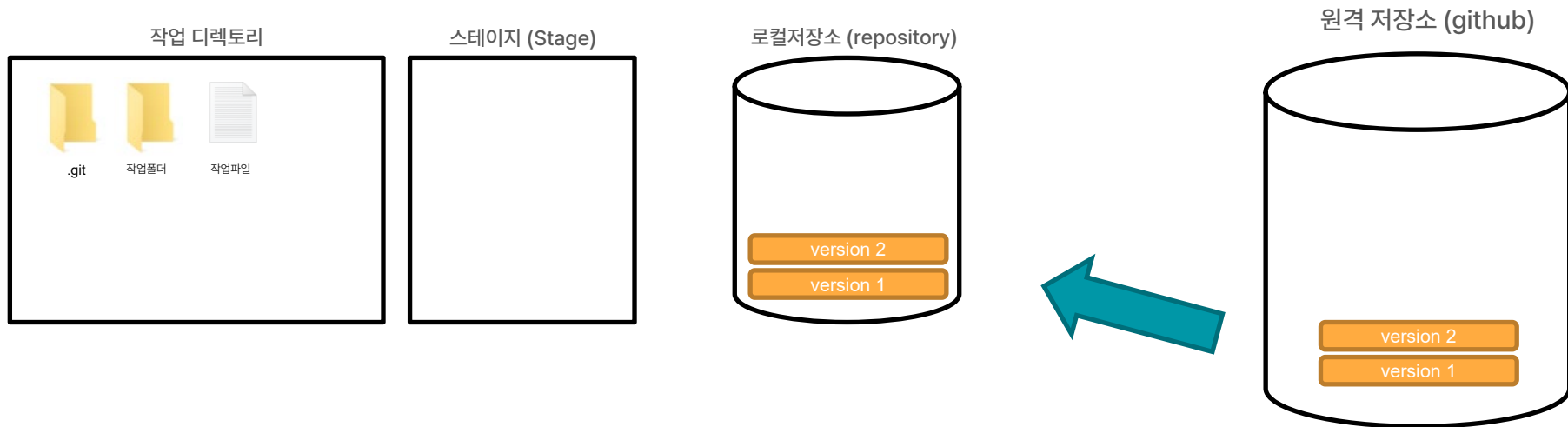
5. Github를 활용한 협업

- local vs. remote
 - 원격 저장소의 장점 : 백업 & 협업



5. Github를 활용한 협업

- 원격 저장소를 복제해오기
 - `git clone` [원격저장소경로]

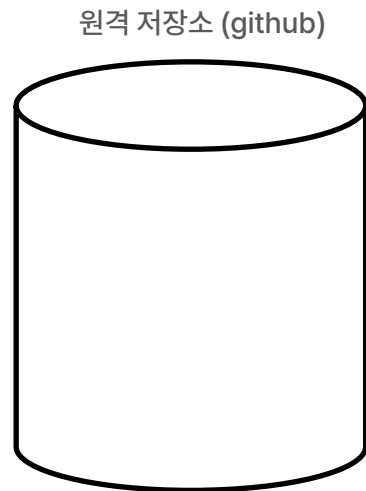
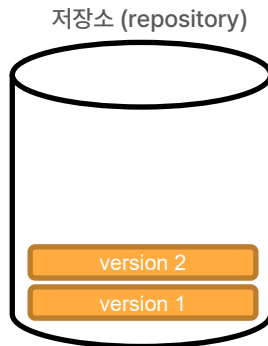
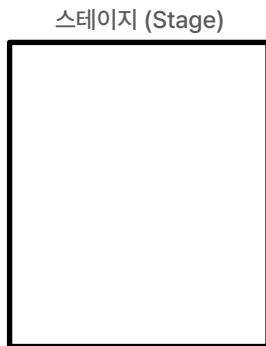


5. Github를 활용한 협업

- 원격 저장소 관련 설정
 - 원격 저장소 추가하기
 - `$ git remote add origin git@github.com:abc/abc.git`
 - 원격저장소이름 origin 을 추가함
 - git clone 을 하면 자동으로 설정됨. clone 없이 설정하거나, 저장소 주소, 이름 바뀔 때 필요
 - 원격 저장소 조회하기
 - `$ git remote -v`
 - 원격 저장소 이름 변경하기 (origin -> changed)
 - `$ git remote rename origin changed`
 - 원격 저장소 삭제하기 (changed 삭제)
 - `$ git remote remove changed`
 - 원격 저장소 주소 변경하기
 - `$ git remote set-url origin [.git 으로 끝나는 url]`

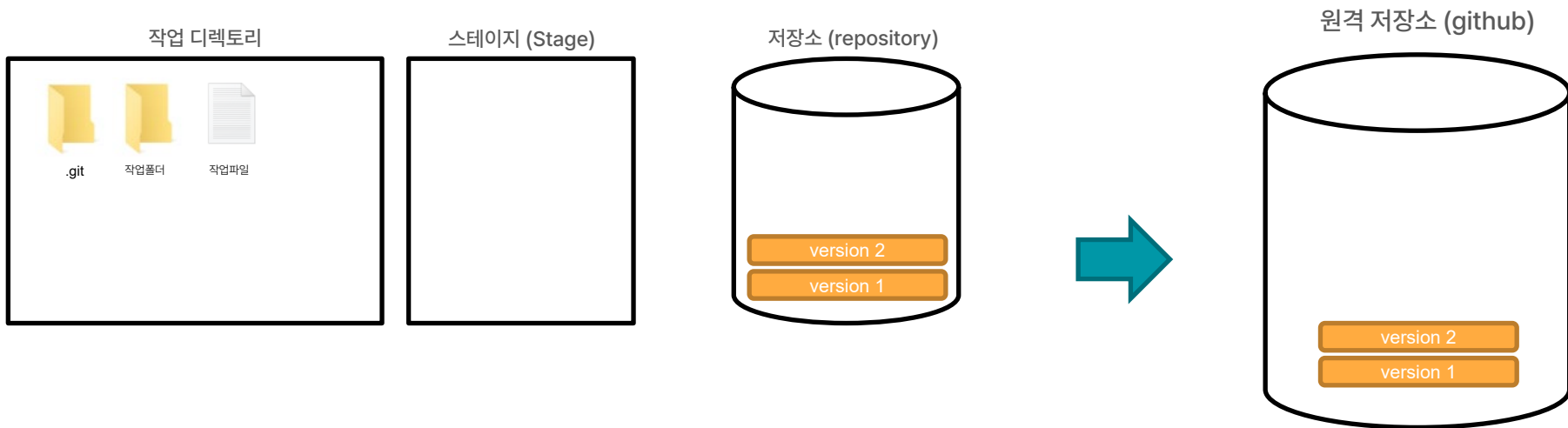
5. Github를 활용한 협업

- 로컬 저장소를 원격 저장소(remote)와 동기화



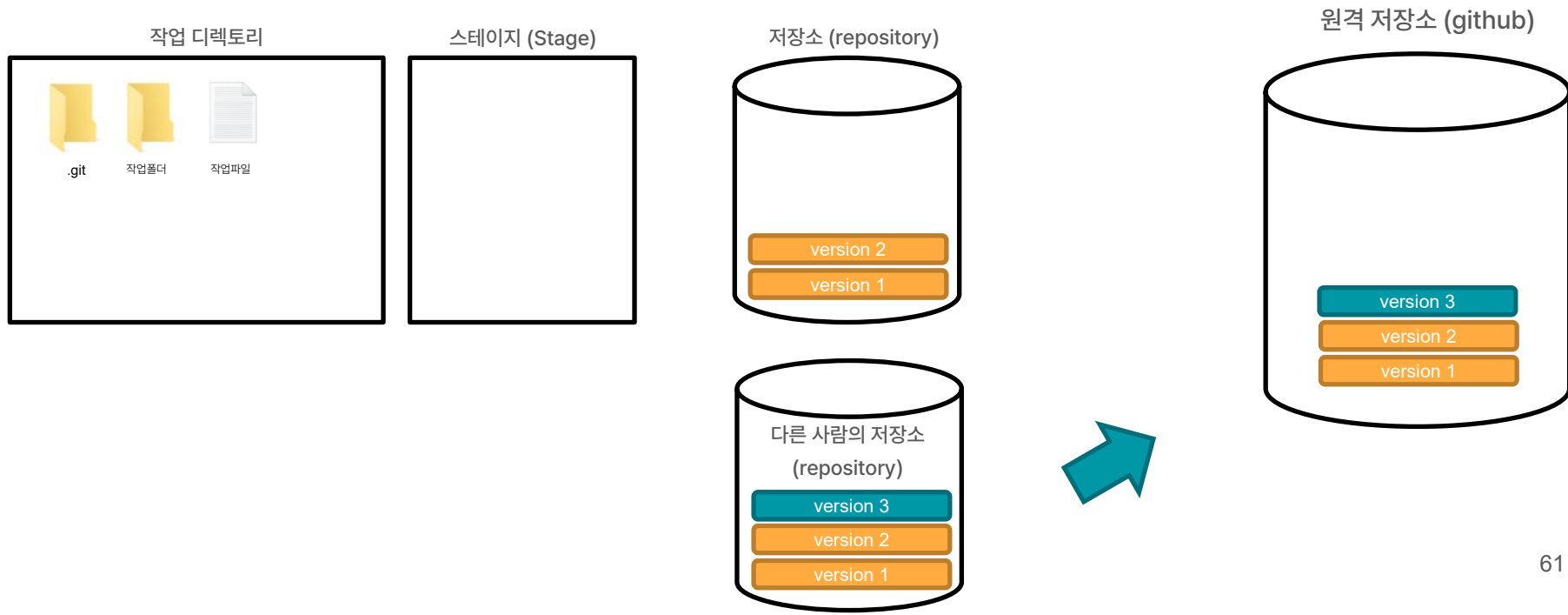
5. Github를 활용한 협업

- 로컬 저장소를 원격 저장소(remote)와 동기화
 - \$ git push [원격저장소이름] [브랜치이름] : 로컬 저장소의 브랜치를 원격 저장소로 밀어넣음
 - 최초 실행 시 -u 옵션을 주면 그 이후는 git push 만으로 간략하게 명령 실행 가능



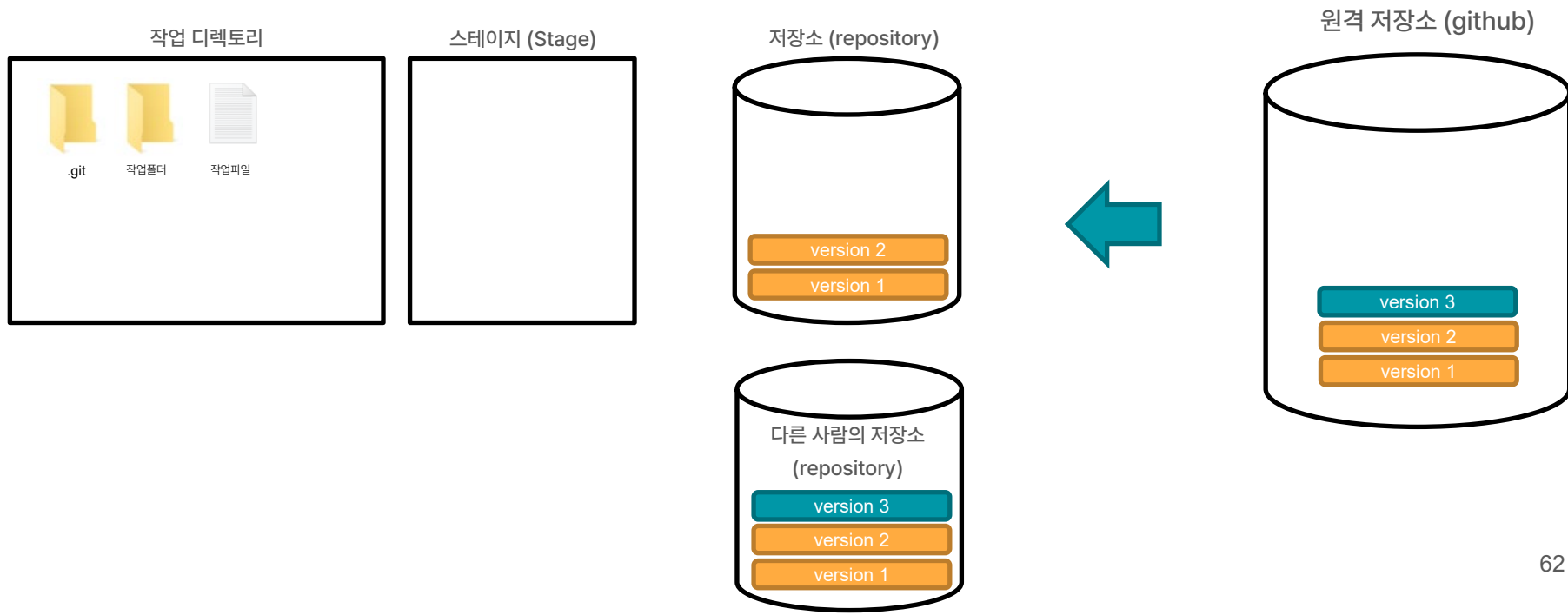
5. Github를 활용한 협업

- 원격 저장소(remote)를 로컬 저장소와 동기화



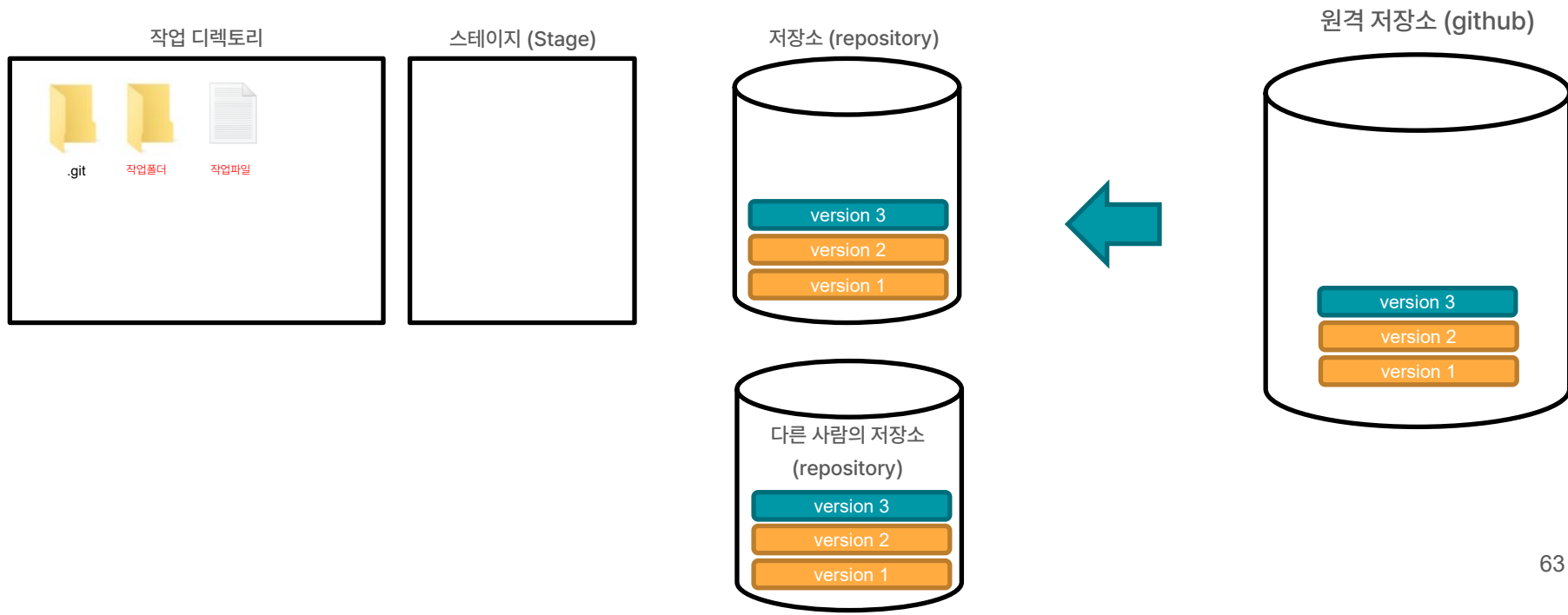
5. Github를 활용한 협업

- 원격 저장소(remote)를 로컬 저장소와 동기화
 - \$ git pull [원격저장소이름] [브랜치이름] : 원격 저장소의 commit 내역을 로컬 저장소로 당겨옴



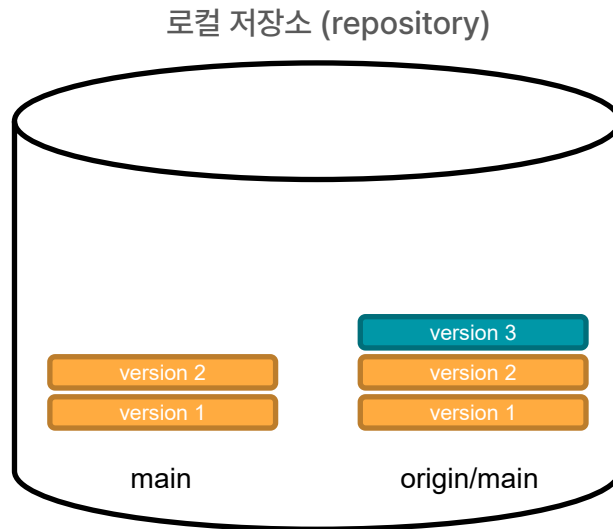
5. Github를 활용한 협업

- 원격 저장소(remote)를 로컬 저장소와 동기화
 - 최신 버전으로 파일들을 자동으로 업데이트



5. Github를 활용한 협업

- 원격 저장소(remote)를 로컬 저장소와 동기화
 - 바로 병합하지 않고, 별도 branch 로만 존재시킬 수 있음
 - `$ git fetch [원격저장소명]`
 - `git status` 를 확인하면 1 commit 뒤에 있음을 경고해줌
 - 따라서 `git merge origin/main` 을 하면 병합됨
 - `git pull = git fetch + git merge`



5. Github를 활용한 협업

- remote repository 에 commit 기록이 없는 경우
 - clone 할 수 없음
 - 해결 방안
 1. 임의의 파일을 github에서 만들어서 첫번째 commit 기록을 생성한 뒤 clone 수행
 2. 로컬에서 작업한 폴더에서 원격 저장소 경로를 추가한 후 push 수행

...or create a new repository on the command line

```
echo "# test2" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:kw-ic-oss/test2.git
git push -u origin main
```



...or push an existing repository from the command line

```
git remote add origin git@github.com:kw-ic-oss/test2.git
git branch -M main
git push -u origin main
```



5. Github를 활용한 협업

- 누구나 푸시할 수 있다면, 여러 문제가 발생할 수 있음
 - linux kernel 을 누구나 푸시할 수 있었다면..?
- Pull Request 를 통한 협업
 - 원격 저장소가 내 변경 사항을 pull 하도록 request 하는 방식의 협업
 - 제가 당신 코드를 이렇게 변경하고 싶은데, 이 변경을 검토해서 유용하다고 생각이 들면 당신의 코드에도 반영해주세요
 - 오픈소스에 기여하는 방법!
 - 규모가 큰 오픈소스 소프트웨어에 contribute 할 수 있다면, 실력을 인정받은 것이나 다름 없음

5. Github를 활용한 협업

- Pull Request 방법
 - 기여하려는 repo 를 본인 계정으로 fork 하기
 - fork 한 repo 를 local 환경으로 clone 하기
 - branch 생성한 후 새 branch에서 작업 시작하기
 - 작업한 branch 본인 계정의 repo 로 push 하기
 - 원작자에게 pull request 보내기
 - 원작자는 pull request 를 검토하여 반영할지 말지 결정



You need to fork this repository to propose changes.

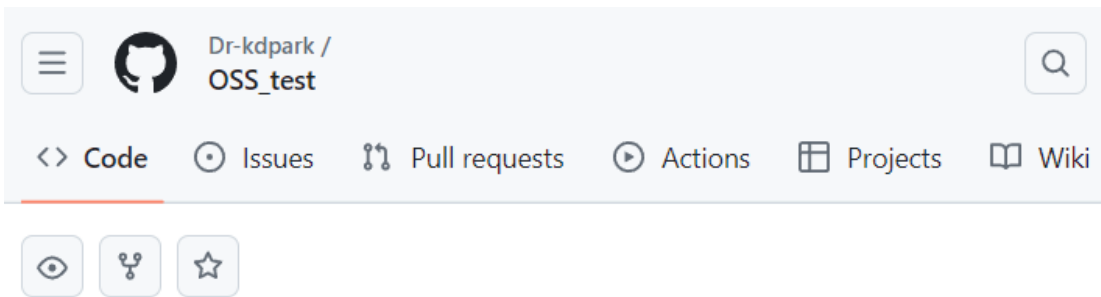
Sorry, you're not able to edit this repository directly—you need to fork it and propose your changes from there instead.

Fork this repository

[Learn more about forks](#)

5. Github를 활용한 협업

- 팀 내에서 협업을 할 때 Pull Request 활용하기
 - 문제가 있는 issue 를 발행함
 - issue 명으로 branch 만들어서 issue와 연결하기
 - branch 에서 작업 완료 후 push, pull request 보냄
 - 팀장은 pull request 를 검토하여 반영함
 - issue 가 자동으로 close 됨



Thank you

Q & A